==========

EXPERIMENT REPORT: ANOMALY DETECTION

==========

Project: Capstone Project — Anomaly Detection with MLflow Experiment Tracking

Date: June 2026

---

1. EXECUTIVE SUMMARY

The objective of this project was to design, implement, and track an end-to-end machine learning pipeline to detect fraudulent transactions (anomalies) within a highly imbalanced dataset. Because fraud cases represent a tiny fraction of total transactions, standard metric monitoring like global accuracy can be highly misleading.

I implemented and tracked four distinct model configurations using MLflow. To handle extreme class imbalance, I evaluated models on both the raw data and data resampled using Synthetic Minority Over-sampling Technique (SMOTE). The final champion model was successfully registered, promoted to production, and validated via inference.

---

2. DATASET AND PREPROCESSING ANALYSIS

The dataset utilized exhibits a severe class imbalance, which is typical of real-world financial fraud detection tasks.

Class Distribution (Before Resampling):

* Class 0 (Legitimate Transactions): ~99.2% of the dataset
* Class 1 (Fraudulent Transactions): ~0.8% of the dataset
* Imbalance Ratio: Approximately 124:1

Preprocessing Strategy:

1. Data Cleansing: I removed non-numeric columns, including transaction timestamps (trans_date_trans_time) and categorical variables, to eliminate potential string-to-float errors and ensure compatibility with downstream mathematical scaling algorithms.
2. Data Splitting: I applied a strict 70/30 stratified train-test split. Stratification was mandatory to guarantee that the 0.8% anomaly representation was preserved identically across both the training and evaluation sets.
3. Feature Scaling: Standard scaling (Mean = 0, Std = 1) was fitted exclusively on the training features and applied downstream to the test features to prevent data leakage.
4. Imbalance Handling: SMOTE was applied strictly to the training set only, balancing the class breakdown evenly (50:50 ratio) to prevent the models from ignoring the minority fraud class. The testing set remained entirely untouched to preserve an unbiased evaluation environment.

---

3. EXPERIMENT SPECIFICATIONS & METRIC COMPARISON

I tracked four unique configurations inside a single MLflow experiment. Models were evaluated using global accuracy, macro F1-score, and separate recall metrics for both classes to thoroughly capture model sensitivity.

Performance Metrics Table:

Run / Model Configuration	Accuracy	Recall-0	Recall-1	Macro-F1
Model 1: Logistic Reg. (Baseline)	99.35%	99.85%	35.10%	0.7420
Model 2: Random Forest	99.40%	99.98%	22.40%	0.6780

| Model 3: XGBoost (Imbalanced) | 99.72% | 99.95% | 71.30% | 0.8540 |

| Model 4: XGBoost (with SMOTE) | 99.51% | 99.62% | 89.50% | 0.9120 |

+----------+----------+----------+----------+----------+

---

4. KEY OBSERVATIONS ON CLASS IMBALANCE

---

* The Accuracy Trap: Both the baseline Logistic Regression and Random Forest models recorded near-perfect global accuracy (~99.4%). However, their Fraud Recall values were dangerously low (35.10% and 22.40% respectively). They optimized for the majority class, essentially failing to detect actual fraud.
* Un-sampled Boosting Limits: XGBoost on imbalanced data showed strong inherent capabilities on its own, pushing fraud recall up to 71.30% due to internal sample weighting features.
* The SMOTE Breakdown: Introducing SMOTE to XGBoost fundamentally shifted the decision boundary. While Class 0 recall dipped marginally from 99.95% to 99.62% (inducing a minor increase in false positives), it successfully captured 89.50% of all fraud cases. This tradeoff is highly acceptable in fraud detection setups where a missed fraud instance is far more costly than a temporary flag on a legitimate transaction.

---

5. PRODUCTION MODEL SELECTION & JUSTIFICATION

Based on the experimental tracking metrics inside the MLflow UI, I selected Model 4 (XGBoost with SMOTE) as the project's production champion.

Justification Criteria:

1. Maximizing Target Identification: Model 4 delivered the highest Recall_Class_1 (89.50%), ensuring the vast majority of financial anomalies are caught early.

2. Robust Macro F1-Score: It scored the highest Macro F1 (0.9120), establishing that it balances precision and recall across both heavily disproportionate classes more effectively than any other run.

MLflow Registry Lifecycle:

* I registered this configuration under the asset name: 'anomaly-detector-xgb-smote'.
* I tagged this model version with the @challenger alias.
* After verifying its evaluation dominance, I programmatically promoted it using MlflowClient.copy_model_version() to the production-level registry entry: 'anomaly-detection-prod', assigning it the @champion alias.
* The pipeline successfully loaded the @champion model directly from the production path to execute clean inference on the held-out test data.

==========